SMART STUDY RECALL / ENGINEERING AND RESEARCH GUIDE

Complete Project Working Report

End-to-end architecture, data flow, models, retrieval, evaluation, operations and research limitations

Prepared from the repository implementation and experiment reports as inspected on 27 October 2026.

Faculty presentation edition | Implementation facts are separated from research hypotheses and unvalidated claims.

How to use this document

This guide explains the running implementation, what data enters each component, what algorithms do with it, what is saved, how to operate the project, and what the current experiments do and do not establish. Formulae use plain ASCII notation. Source paths at the end let a reviewer trace each description back to the repository.

Important interpretation rule: a predicted next-session correct fraction is a score proxy. It is not a measured probability that a learner will remember a course concept on a future date.

Contents

01. 1. Project at a glance

02. 2. Complete learner workflow

03. 3. Models and algorithms in the repository

04. 4. Data model, persistence and privacy

05. 5. Evaluation: what the numbers mean

06. 6. Run and demo the project

07. 7. Honest project summary and next research step

1. Project at a glance

Smart Study Recall combines a study activity tracker, material-grounded assistant, experimental next-session score prediction and a revision planner. Its user-facing goal is to help a learner record study, practice retrieval and see a suggested review interval.

It is a three-service local web application: static browser pages, an authenticated FastAPI application API and a separate ML API. Supabase Auth handles sign-in; PostgreSQL stores learner and course records. An optional external language model generates notes, questions and answers from retrieved course passages.

Browser pages HTML/CSS/JS :5500	App API FastAPI :8002	ML API FastAPI :8000	PostgreSQL Supabase database

The browser sends a Supabase bearer token to the app API. The app API validates that token with Supabase Auth and resolves a local profile. The app API reads/writes the database and calls the ML service for candidate-gap predictions. The ML service does not directly access user accounts or database rows; it receives bounded feature payloads.

Component	Responsibility	Main state
Frontend	Navigation, forms, charts, timer, RAG and planner UI	Session token and local-only focus/flashcards
App API	Authentication, access control, CRUD, schedules, materials and RAG orchestration	Profiles, topics, events, schedules, documents, quiz results, chat history
ML API	Predict next-session score for candidate gaps; serve separate KT candidates	Trained model artifacts loaded from ml/models
Supabase	Email sign-in and PostgreSQL database	Accounts plus relational learning data

This codebase is a research prototype. Model output and the 80% review threshold are experimental; the app does not measure a learner's actual biological forgetting time.

2. Complete learner workflow

A. Sign in and establish a profile

A learner signs up or logs in through Supabase Auth from the browser. The frontend stores access and refresh tokens in localStorage. Each protected request sends the current bearer token to the FastAPI app. The backend verifies the user with Supabase and resolves or creates the matching Profile row by Supabase user ID. Ownership checks scope topics and learning records to that profile.

B. Create topics and log study

A topic stores its name and optional subject. A study or quiz result is represented as a LearningEvent with student ID, topic ID, questions seen, questions correct and event time. These event counts are the direct app-domain inputs used by the next-session prediction adapter. The focus timer records elapsed minutes locally; it does not create a learning outcome or recall label.

C. Get a recall estimate and planner action

The app backend orders a topic's learning events and calls the ML API for predictions at a fixed set of candidate gaps. The selected model returns predicted next-session correct fractions. A separate policy chooses a review delay. The Planner can create one upcoming model-based review for eligible topics, while user-created schedules remain available.

D. Upload and study material

A signed-in user uploads PDF, DOCX, TXT or Markdown, subject to the 12 MB limit. Text extraction runs in the backend, overlapping passages are generated and records are stored in PostgreSQL. The Materials and RAG pages can produce cited notes, source questions and chatbot answers. Quiz responses can feed a deterministic concept-gap heuristic in the adaptive RAG retrieval path.

Record session seen/correct/time	Build history ordered by topic	Predict scores for candidate gaps	Apply 80% policy choose delay	Save schedule show reminder

The RAG chatbot follows a different pipeline from the recall predictor: it indexes document chunks, ranks evidence for the question, then optionally asks a configured text-generation provider to write a cited answer. The chatbot does not call the HLR/GRU predictor to generate text.

---

3. Models and algorithms in the repository

Name	What it predicts/does	Status in product
HLR	Next-session correct/seen fraction conditional on a candidate gap	Validation-selected champion used by topic review forecasts
Mean, linear, Random Forest	Baseline predictions for same next-session task	Evaluation comparators
GRU	Next-session score from ordered history of up to 10 events	Evaluated candidate; not selected by current validation-MAE rule
KT MLP	Next-interaction score from current event plus order-agnostic summary	Separate ML API candidate
KT LSTM	Next-interaction score from ordered event sequence	Separate ML API candidate
KT Transformer	Next-interaction score with causal attention over event sequence	Separate ML API candidate
RAG BM25 / TF-IDF / RRF	Ranks source passages for a question	Live retrieval methods
Recall-aware RAG reranker	Adds quiz-gap and recency signals to passage ranking	Live deterministic heuristic; not trained

The app's main prediction experiment is not the same task as the separate KT model experiment. The former pairs a learner-item history with the next observed session of that same learner-item and supplies a candidate gap. The KT task predicts the current event outcome from its known item and pre-event sequence. Neither target is an app-specific delayed recall label.

HLR is selected as the current main serving artifact because it has the lowest mean validation MAE over both evaluation protocols. GRU is better on some test metrics, including reported RMSE/ECE, so the champion choice is objective-dependent. The current rule does not select on forgetting-date error or measured course learning outcomes.

The local model artifacts can be different from the ones loaded by a running ML server. Restart the ML service after changing artifacts. The API health route reports whether the champion artifact exists; existence does not

prove domain validity.

4. Data model, persistence and privacy

Entity	Meaning	Storage
Profile	Supabase account mapping, display name, email and role	PostgreSQL
Topic	Learner's course concept/topic	PostgreSQL
LearningEvent	Study session question counts and event timestamp	PostgreSQL
RevisionSchedule	A planned review timestamp and completion/reschedule state	PostgreSQL
StudyMaterial / Chunk	Uploaded file metadata and extracted text passages	PostgreSQL
StudyQuizSession / Response	Generated questions, scores and concept-level right/wrong outcomes	PostgreSQL
StudyRagInteraction	Persisted user question, answer, retrieval mode, citations, diagnostics and support vote	PostgreSQL after migration 004
Focus minutes / flashcards	Personal focus timer state and flashcard text	Browser localStorage

The data export profile endpoint returns account-linked records. Focus time is deliberately not treated as recall evidence. Uploaded documents and chat contain potentially sensitive study information; use minimization, access control, consent and institutional review before exporting or reusing records for research.

Database changes are managed through SQL files in migrations/. The materials schema is added by migration 003; persistent RAG interaction history is added by migration 004. Existing data does not appear in a new schema automatically: run the appropriate SQL in Supabase before using new storage-backed routes.

5. Evaluation: what the numbers mean

The next-session report uses 617 sampled learners and 62,931 eligible examples from the Duolingo Learning Traces source. Learners are selected by deterministic hash sampling at 1-in-100. It evaluates both a chronological future split and a learner-disjoint split, with three seeds and learner-cluster bootstrap intervals.

The separate KT forward-time comparison uses a different 1-in-100 sample: 119,140 event rows and 1,118 learners, one seed. Its learner-held-out comparison is a smaller 1-in-500 sample with 22,670 rows and 211 learners. Never compare their raw metric rows as though they used the same sample and split.

Metric	Formula / interpretation	Caution
MAE	Mean absolute difference between predicted and observed score	Easy to read in score-fraction units
RMSE	Square root of mean squared prediction error	Penalizes large errors more strongly
ECE	Weighted average gap between mean score prediction and outcome within 10 score bins	Bin and sampling dependent; not full calibration proof
Brier score	Mean squared error for probability-like prediction on binary outcomes, here adapted/weighted for aggregate data	Target is fractional session score in KT runs

Metric	Formula / interpretation	Caution
AUC	Ranking discrimination for binary-expanded/weighted evaluation	Not a measure of calibrated magnitude
Bootstrap 95% CI	Resample learner clusters 300 times and recompute metric	Uncertainty applies to the sampled evaluation design

No judged RAG relevance set, human-reviewed answer correctness set, app-domain delayed assessment, or randomized adaptive-versus-fixed learning study exists in this repository. Therefore RAG accuracy, app-domain forgetting-date accuracy, causal learning improvement and patent novelty are not established results.

When presenting results, state the task, data, sample, split and seed next to each metric. Do not present a Duolingo session score as a validated course-topic forgetting probability.

---

6. Run and demo the project

The local development setup requires three separate processes. Use a separate terminal for each. Backend configuration is read from the repository-root .env; frontend Supabase values and API_URL are in frontend/js/config.js. Never put a database password, service-role key or AI API key in browser JavaScript.

```
# Terminal 1: from repository root
.\.venv\Scripts\python.exe -m uvicorn app.main:app --host 127.0.0.1 --port 8002 --reload

# Terminal 2: from repository root
Set-Location .\ml\src
..\..\.venv\Scripts\python.exe -m uvicorn api:app --host 127.0.0.1 --port 8000

# Terminal 3: VS Code Live Server
Serve frontend/ at http://127.0.0.1:5500/index.html
```

Before the demo, apply database migrations in order; confirm /health, /health/database and the ML /health endpoint; sign in; create a topic; log at least one study session; open Dashboard and Planner; upload a text-based source; ask a source-grounded question; inspect citations; and generate a quiz. If a model is unavailable, explain the explicit fallback instead of claiming the model answered.

Good faculty demo narrative: show one topic's input events, explain the prediction target, show the candidate score curve, show the 80% review policy selecting a date, then show where that schedule is persisted. Separately upload a short course handout, trace extraction to chunk storage to retrieval to cited generation, and distinguish the retrieval strategy from the configured language model.

---

7. Honest project summary and next research step

The project integrates standard web application engineering, retrieval-augmented study support and experimental sequence/recall modeling. Its strongest technical explanation is the end-to-end data path and transparent separation between a score predictor and a scheduling policy. Current evidence supports software functionality and offline evaluation on a public language-learning proxy, not validated memory science.

A defensible next study should collect consented course-topic session histories and delayed quizzes, compare fixed scheduling against the current adaptive policy, hold out learners and future time, pre-register a primary outcome, report participant-level intervals, and avoid using the test set to select the model.

A patent or novelty claim requires a separate literature and patent search and a legally informed review. Combining a known half-life model, a RAG chatbot and a planner is not by itself proof of patentability.

Repository traceability and source files

The report describes the local repository snapshot, not an independently deployed cloud instance. A running deployment can differ if its backend, database migrations, model artifacts or environment variables have not been updated.

Primary source files reviewed:

- README.md
- docs/IMPLEMENTATION_AUDIT.md
- docs/MODEL_ARCHITECTURE.md
- docs/DATASET_AND_PREPROCESSING.md
- docs/EXPERIMENT_PROTOCOL.md
- docs/RESULTS.md
- docs/RESEARCH_LIMITATIONS.md
- docs/API.md
- app/main.py
- app/models.py
- app/routers/
- app/services/
- ml/src/
- ml/reports/